DOCUMENT 2

Mathematics for AI

The Complete One-Pass Refresher Guide

AI Engineer Placement Preparation Series

Generated: February 2026 | Cutting-Edge Edition

Table of Contents

Why This Matters

Math is the language AI speaks internally. When you call model.fit(), hundreds of matrix multiplications, derivatives, and probability calculations happen under the hood. You can use AI without math the same way you can drive without understanding engines. But when the car breaks down, the mechanic knows what to do and you don't.

In AI Engineer interviews, math questions serve a specific purpose: they test whether you understand WHY things work, not just HOW to use them. When an interviewer asks about eigenvalues, they're really asking if you understand dimensionality reduction. When they ask about gradients, they want to know if you can debug a model that won't converge.

This document covers the **exact math you need** for AI, nothing more. Every concept is explained through the lens of "where does this show up in AI?" If a math topic doesn't appear in ML/DL, it's not here. If it appears everywhere, it gets extra depth.

Cutting-Edge Note (2025-2026)

Interviews are shifting from "prove you know the math" to "show you can reason with math." You'll rarely be asked to solve an integral. You WILL be asked: "Why does your model overfit?" (answer requires understanding bias-variance, norms, regularization). "Why does cosine similarity work for embeddings?" (answer requires understanding vector spaces, dot products). "Why does temperature control LLM creativity?" (answer requires understanding probability distributions, softmax).

1. Linear Algebra

Linear algebra is the single most important branch of math for AI. Neural networks are essentially chains of matrix multiplications with nonlinearities sprinkled in. If you understand matrices, you understand 70% of what a neural network does.

1.1 Scalars, Vectors, Matrices, and Tensors

These are just different "sizes" of numbers. A **scalar** is a single number (like 5). A **vector** is a list of numbers (like [3, 7, 2]). A **matrix** is a grid of numbers (rows and columns). A **tensor** is the general term for any of these, and can have any number of dimensions.

Type	Shape Example	AI Example
Scalar	Just a number	Learning rate (0.001), loss value (2.34)
Vector	[n] (1D array)	A single word embedding (e.g., 768 numbers representing "cat")
Matrix	[m x n] (2D)	Weight matrix in a neural network layer, batch of embeddings
3D Tensor	[batch x seq x dim]	A batch of sentences, each with multiple word embeddings
4D Tensor	[batch x channels x H x W]	A batch of RGB images

In Python/PyTorch: Everything is a tensor. Even a scalar is a 0-dimensional tensor. `torch.tensor(5)` is a scalar, `torch.tensor([1,2,3])` is a vector, `torch.randn(3,4)` is a 3x4 matrix.

1.2 Vector Operations

Dot Product

The **dot product** of two vectors a and b multiplies corresponding elements and sums them: *`a . b = a1*b1 + a2*b2 + ... + an*bn`*. It produces a single number (scalar).

Why it matters in AI: The dot product measures how "similar" two vectors are. This is the foundation of attention mechanisms in Transformers, similarity search in vector databases, and recommendation systems. When you compute Query . Key in attention, that's a dot product.

Cosine Similarity

Cosine similarity normalizes the dot product by the magnitudes of the vectors: *`cos(a, b) = (a . b) / (||a|| * ||b||)`*. The result ranges from -1 (opposite) to 1 (identical direction), with 0 meaning orthogonal (unrelated).

Why it matters in AI: This is THE similarity metric for embeddings. When you search a vector database (Pinecone, ChromaDB, FAISS), it uses cosine similarity to find the most relevant documents. RAG systems live and die by this metric.

> **Interview Hot Topic: Cosine Similarity in RAG/Embeddings**

"Why do we use cosine similarity instead of Euclidean distance for text embeddings?" Answer: Cosine similarity measures direction (meaning), not magnitude (length). Two documents about the same topic might have different lengths, producing vectors of different magnitudes. Cosine similarity ignores this and focuses on whether they point in the same direction in embedding space.

Norms (L1 and L2)

A **norm** measures the "size" or "length" of a vector. The two most common are:

Norm	Formula	Intuition	AI Use
L1 (Manhattan)	|x1| + |x2| + ... + |xn|	Sum of absolute values. Distance if you can only walk along grid lines.	L1 regularization (Lasso). Encourages sparse weights (drives some to exactly 0).
L2 (Euclidean)	sqrt(x1^2 + x2^2 + ... + xn^2)	Straight-line distance. The "normal" distance you think of.	L2 regularization (Ridge). Penalizes large weights evenly. Also: vector magnitude in cosine sim.

1.3 Matrix Operations

Matrix Multiplication

If A is [m x n] and B is [n x p], then A*B is [m x p]. The rule: **inner dimensions must match**. Each element (i, j) in the result is the dot product of row i from A and column j from B.

Why it matters in AI: Every layer of a neural network is a matrix multiplication: *`output = input @ weights + bias`*. A batch of 32 samples with 784 features, passing through a layer with 256 neurons = [32 x 784] @ [784 x 256] = [32 x 256]. This is literally what happens when you call `nn.Linear(784, 256)` in PyTorch.

Transpose

The **transpose** of matrix A (written A^T) flips rows and columns. If A is [m x n], A^T is [n x m].

Where you see this: Everywhere. In attention: Q * K^T computes all pairwise similarities. In backpropagation: gradients are computed using transposed weight matrices. In data: often you need to transpose your data to get rows=samples, columns=features.

Inverse

The **inverse** of a square matrix A (written A^-1) is the matrix such that A * A^-1 = I (identity matrix). Not all matrices have inverses. If a matrix doesn't have one, it's called **singular**.

AI relevance: Used in closed-form linear regression (Normal Equation): *`w = (X^T X)^-1 X^T y`*. In practice, we rarely compute inverses directly because it's computationally expensive. We use gradient descent instead.

1.4 Eigenvalues and Eigenvectors

For a square matrix A, an **eigenvector** v is a special vector that, when multiplied by A, only gets scaled (not rotated): *`Av = lambda * v`*. The scaling factor lambda is the **eigenvalue**.

Intuition: Think of a matrix as a transformation. Most vectors change direction when transformed. Eigenvectors are the special directions that stay the same, they only stretch or shrink. The eigenvalue tells you how much.

AI relevance: PCA (Principal Component Analysis) finds the eigenvectors of the covariance matrix. The eigenvector with the largest eigenvalue points in the direction of maximum variance in your data. This is how PCA reduces dimensions while preserving the most information.

1.5 Singular Value Decomposition (SVD)

SVD decomposes ANY matrix (not just square ones) into three matrices: `A = U * S * V^T` where U and V are orthogonal matrices and S is a diagonal matrix of **singular values** (always positive, sorted largest to smallest).

Why it matters: SVD is the mathematical backbone of dimensionality reduction, latent semantic analysis, recommender systems (Netflix Prize used SVD), and image compression. The singular values tell you how much "information" each dimension carries. By keeping only the top k singular values, you get the best rank-k approximation of your data.

Connection to PCA: PCA is essentially SVD applied to centered data. The right singular vectors (V) are the principal components.

2. Calculus

Calculus in AI boils down to one core idea: how do you find the minimum of a function? That's what training a model is. You have a loss function, and you want to find the parameters (weights) that minimize it. Calculus gives you the tools to do this efficiently.

2.1 Derivatives

The **derivative** of a function f(x) at a point tells you the **rate of change** (slope) at that point. If the derivative is positive, the function is increasing. If negative, it's decreasing. If zero, you're at a peak, valley, or flat point.

AI connection: If the derivative of your loss with respect to a weight is positive, it means increasing that weight increases the loss. So you should DECREASE the weight. That's gradient descent in one sentence.

Key Derivative Rules You Need

Rule	Formula	Example
Power Rule	d/dx [x^n] = n * x^(n-1)	d/dx [x^3] = 3x^2
Chain Rule	d/dx [f(g(x))] = f'(g(x)) * g'(x)	d/dx [sin(3x)] = cos(3x) * 3
Product Rule	d/dx [f*g] = f'g + fg'	d/dx [x * e^x] = e^x + x*e^x
Exponential	d/dx [e^x] = e^x	Appears in softmax, sigmoid
Log	d/dx [ln(x)] = 1/x	Appears in cross-entropy loss

2.2 Partial Derivatives and Gradients

When a function has multiple inputs (like f(x, y, z)), a **partial derivative** is the derivative with respect to one variable while treating the others as constants. The **gradient** is the vector of ALL partial derivatives: `grad(f) = [df/dx, df/dy, df/dz]`

Key insight: The gradient points in the direction of steepest increase. So to minimize a function, you move in the OPPOSITE direction of the gradient. This is exactly what gradient descent does.

In a neural network: If your model has 10 million parameters, the gradient is a vector of 10 million numbers. Each number tells you "if you nudge this specific weight a tiny bit, how much does the loss change?"

2.3 The Chain Rule (Backbone of Backpropagation)

The **chain rule** is THE most important calculus concept for deep learning. It says: if `y = f(g(x))`, then `dy/dx = dy/dg * dg/dx`. In words: multiply the derivatives along the chain.

Why it's everything in deep learning: A neural network is a chain of functions. Input goes through layer 1, then layer 2, then layer 3, etc. To compute how the loss changes with respect to a weight in layer 1, you need to chain the derivatives through every subsequent layer. This is literally what backpropagation does, it applies the chain rule from the output back to the input, layer by layer.

Interview Favorite: Explain Backpropagation

Backpropagation = chain rule applied systematically. Forward pass: compute the output and loss. Backward pass: starting from the loss, compute gradients layer by layer going backwards, using the chain rule at each step. Each layer's gradient depends on the gradient from the layer above it (hence "back" propagation). These gradients tell the optimizer how to update each weight.

2.4 Jacobian and Hessian (High-Level Awareness)

The **Jacobian** is a matrix of all partial derivatives for a vector-valued function. If f maps R^n to R^m, the Jacobian is [m x n]. It generalizes the gradient to functions with multiple outputs.

The **Hessian** is a matrix of second-order partial derivatives. It tells you about the curvature of a function (not just the slope). Useful for understanding whether a critical point is a minimum, maximum, or saddle point.

AI relevance: You rarely compute these directly, but they appear conceptually. Second-order optimizers (like Newton's method, L-BFGS) use Hessian information. The reason Adam optimizer works well is that it approximates some second-order information cheaply. Understanding curvature also explains why loss landscapes are hard to navigate in high dimensions.

3. Probability and Statistics

Probability is the math of uncertainty. AI models don't output certainties; they output probabilities. A classifier says "87% chance this is a cat." An LLM says "the next token is 'the' with probability 0.23." Understanding probability is understanding what your model is actually telling you.

3.1 Core Probability Concepts

Random Variables

A **random variable** is a variable whose value is determined by a random process. It can be **discrete** (finite outcomes, like rolling a die) or **continuous** (any value in a range, like someone's height).

Probability Distributions

A distribution describes how likely each outcome is. Here are the ones you MUST know:

Distribution	Type	What It Models	AI Use Case
Bernoulli	Discrete	Single yes/no trial (coin flip)	Binary classification output
Binomial	Discrete	Number of successes in n trials	Number of correct predictions in a batch
Poisson	Discrete	Count of events in a fixed interval	Number of anomalies per time window
Uniform	Continuous	All outcomes equally likely	Random weight initialization
Gaussian (Normal)	Continuous	Bell curve, defined by mean and std dev	The most important distribution. Noise, errors, Batch Norm, VAEs, everywhere.
Categorical	Discrete	One of k possible outcomes	Multiclass classification (softmax output)

Expectation and Variance

Expectation (mean): The average value of a random variable. For discrete: `E[X] = sum(x_i * P(x_i))`. It's the "center" of the distribution.

Variance: How spread out the values are from the mean: `Var(X) = E[(X - E[X])^2]`. **Standard deviation** is just sqrt(Variance), in the same units as the data.

AI relevance: Batch normalization computes the mean and variance of each batch to normalize activations. Variance in predictions tells you about model uncertainty. Low variance = confident, high variance = uncertain.

3.2 Bayes' Theorem

Bayes' Theorem: `P(A|B) = P(B|A) * P(A) / P(B)`

In words: the probability of A given that B happened equals (the probability of B given A) times (the prior probability of A), divided by (the probability of B). This lets you **update your beliefs** when you get new evidence.

AI connection: Naive Bayes classifier is directly based on this. Bayesian inference is a whole paradigm of ML. More practically: when an LLM generates text, it's computing P(next_token | all_previous_tokens), which is fundamentally a conditional probability.

3.3 Conditional Probability and Independence

`P(A|B)` is the probability of A given B has occurred. Two events are **independent** if `P(A|B) = P(A)`, meaning knowing B happened doesn't change the probability of A.

Why this matters: The "Naive" in Naive Bayes assumes all features are independent given the class. This is almost never true in practice, but the algorithm works surprisingly well anyway. Understanding conditional independence helps you know when this assumption breaks.

3.4 Maximum Likelihood Estimation (MLE)

MLE answers the question: "Given the data I've observed, what parameters make this data most likely?" You choose parameters that maximize `P(data | parameters)`.

Example: If you flip a coin 100 times and get 73 heads, MLE estimates P(heads) = 0.73. That's the value that makes your observed data most probable.

AI connection: Training a neural network by minimizing cross-entropy loss IS maximum likelihood estimation. When you minimize cross-entropy, you're maximizing the likelihood of the training data under your model. They're mathematically equivalent.

3.5 Covariance and Correlation

Covariance measures whether two variables move together. Positive = they increase together. Negative = one increases as the other decreases. Zero = no linear relationship.

Correlation is normalized covariance (range -1 to 1). It's covariance divided by the product of standard deviations. Easier to interpret because it's scale-independent.

AI use: Feature selection uses correlation to remove redundant features. The covariance matrix is the input to PCA. If two features have correlation near 1.0, they carry almost the same information, drop one.

3.6 Central Limit Theorem (CLT)

The **Central Limit Theorem** says: if you take the average of many independent random samples, that average will be approximately normally distributed, regardless of the original distribution. The more samples, the more normal.

Why this matters: This justifies why Gaussian distributions appear everywhere in ML. Training loss averaged over batches is roughly normal. Estimation errors are roughly normal. This is also why A/B testing works, you can compute confidence intervals assuming normality.

4. Optimization

Training an AI model = optimization. You have a loss function (how wrong your model is) and you want to find the parameters that minimize it. This section covers how that actually works.

4.1 Gradient Descent

The core algorithm: **1)** Compute the gradient (direction of steepest increase). **2)** Move a small step in the opposite direction. **3)** Repeat. The formula: `w_new = w_old - learning_rate * gradient`

Variant	How It Works	Pros	Cons
Batch GD	Gradient from ALL training data	Stable, accurate gradient	Very slow for large datasets
Stochastic GD (SGD)	Gradient from ONE sample	Fast updates, can escape local minima	Very noisy, unstable
Mini-Batch GD	Gradient from a small batch (32, 64, 128)	Best of both worlds	Most commonly used in practice

4.2 Learning Rate

The **learning rate** controls how big each step is. Too large = you overshoot the minimum and diverge. Too small = training takes forever and may get stuck. It's often the single most important hyperparameter to tune.

Learning rate schedules change the learning rate during training. Common approaches: start high and decay (step decay), use cosine annealing (smooth decrease and increase), or use warmup (start very low, increase, then decrease).

4.3 Modern Optimizers

Optimizer	Key Idea	When to Use
SGD + Momentum	Adds "velocity" to updates, like a ball rolling downhill	CNNs, when you want fine control and have time to tune
AdaGrad	Adapts learning rate per-parameter (shrinks for frequent features)	Sparse data (NLP with rare words)
RMSProp	Like AdaGrad but uses moving average (doesn't shrink to zero)	RNNs, when AdaGrad decays too fast
Adam	Combines momentum + adaptive rates. Maintains per-param moving averages of gradient and squared gradient.	Default choice for most tasks. Use this unless you have a reason not to.
AdamW	Adam but with proper weight decay (decoupled from gradient)	Transformers, LLM fine-tuning. The current standard for NLP.

Interview Insight: Why Adam Works So Well

Adam adapts the learning rate for each parameter individually. Parameters with large, consistent gradients get smaller learning rates (they're already moving fast). Parameters with small, noisy gradients get larger learning rates (they need a bigger push). This is why Adam converges faster than plain SGD on most problems. The catch: Adam can generalize worse than well-tuned SGD on some image tasks.

4.4 Convex vs. Non-Convex

A **convex function** has a single global minimum (like a bowl). Gradient descent is guaranteed to find it. Linear regression's loss is convex.

A **non-convex function** has multiple local minima and saddle points (like a mountain range). Neural network loss functions are non-convex. Gradient descent is NOT guaranteed to find the global minimum, but in practice, it finds "good enough" minima. Research shows that in high dimensions, most local minima are actually pretty close in value to the global minimum.

5. Information Theory

This section is often missing from standard guides, but is increasingly tested in AI interviews. Information theory provides the mathematical framework behind cross-entropy loss, KL divergence, and understanding why LLMs work the way they do.

5.1 Entropy

Entropy measures the "uncertainty" or "surprise" in a probability distribution: `H(X) = -sum(p(x) * log2(p(x)))`

Intuition: A fair coin has maximum entropy (1 bit), you're maximally uncertain about the outcome. A biased coin (99% heads) has low entropy, the outcome is almost certain. A coin that always lands heads has zero entropy.

AI connection: When an LLM generates text, entropy tells you how "uncertain" the model is about the next token. High entropy = many plausible next tokens. Low entropy = the model is confident about what comes next.

5.2 Cross-Entropy

Cross-entropy measures the difference between two probability distributions (true distribution p and predicted distribution q): `H(p, q) = -sum(p(x) * log(q(x)))`

Why it's THE loss function for classification: Cross-entropy penalizes confident wrong predictions heavily. If the true label is class A and your model predicts P(A) = 0.01 (very wrong), the loss is -log(0.01) = 4.6 (very high). If your model predicts P(A) = 0.99 (correct), the loss is -log(0.99) = 0.01 (very low). This is exactly the behavior we want.

5.3 KL Divergence

KL Divergence (Kullback-Leibler) measures how much one distribution differs from another: `KL(p || q) = sum(p(x) * log(p(x)/q(x)))`

Key properties: KL(p||q) >= 0 always. KL(p||q) = 0 only when p = q. It's NOT symmetric: KL(p||q) is not equal to KL(q||p).

AI uses: Variational Autoencoders (VAEs) minimize KL divergence between the learned latent distribution and a standard normal. Knowledge distillation uses KL divergence to match a student model's outputs to a teacher model's. LLM training also minimizes a form of KL divergence between the model's distribution and the true data distribution.

5.4 Temperature in LLMs

Temperature is a parameter that controls the "sharpness" of a probability distribution. In the softmax function: `softmax(x_i / T)` where T is temperature.

Temperature	Effect	Distribution Shape	Use Case
T = 0 (or very low)	Picks the highest probability token almost always	Very peaked (nearly deterministic)	Factual Q&A, code generation
T = 1.0	Uses the model's raw	Normal distribution	Default, balanced

Temperature	Effect	Distribution Shape	Use Case
	probabilities		output
T > 1.0 (e.g., 1.5)	Flattens the distribution, making unlikely tokens more probable	Spread out (more uniform)	Creative writing, brainstorming

Interview Question: "Why does temperature work?"

Dividing logits by T>1 before softmax compresses the values, making the distribution more uniform (higher entropy = more randomness). Dividing by T<1 amplifies differences, making the distribution sharper (lower entropy = more deterministic). This is pure information theory applied to LLM decoding.

6. Graph Theory Basics

This is an edge topic that gives you a competitive advantage in 2025-2026 interviews. With the rise of agentic AI (LangGraph, multi-agent systems) and Graph Neural Networks, basic graph theory is no longer optional.

6.1 What is a Graph?

A **graph** is a collection of **nodes** (vertices) connected by **edges**. Graphs can be **directed** (edges have direction, like Twitter follows) or **undirected** (mutual connections, like Facebook friendships). Edges can have **weights** (representing distance, cost, or strength).

6.2 Key Concepts

Concept	Definition	AI Relevance
Adjacency Matrix	A matrix where entry (i,j) = 1 if there's an edge from node i to j	Input representation for Graph Neural Networks
Degree	Number of edges connected to a node	Used in graph normalization for GNNs
Path	A sequence of edges connecting two nodes	Agent reasoning traces in LangGraph
Cycle	A path that starts and ends at the same node	Agentic loops (ReAct pattern: reason-act-observe-repeat)
DAG (Directed Acyclic Graph)	Directed graph with no cycles	Computation graphs in PyTorch, ML pipelines
Tree	Connected graph with no cycles	Decision trees, hierarchical clustering

6.3 Why Graphs Matter for Modern AI

1. Agent workflows: LangGraph (the leading agentic AI framework) models agent behavior as a state graph. Nodes are actions, edges are transitions. Understanding graph traversal helps you design better agents.

2. Knowledge Graphs: RAG systems increasingly use knowledge graphs alongside vector databases. Entities are nodes, relationships are edges. GraphRAG combines graph traversal with LLM generation for more accurate retrieval.

3. GNNs: Graph Neural Networks apply deep learning directly to graph-structured data (social networks, molecules, recommendation systems). They use message passing between nodes, which requires understanding adjacency and neighborhoods.

4. Computation graphs: PyTorch autograd builds a DAG of operations during the forward pass, then traverses it backwards for gradient computation. Understanding this helps you debug training issues.

7. Key Comparison Tables

L1 vs L2 Regularization

Feature	L1 (Lasso)	L2 (Ridge)
Penalty term	|w| (sum of absolute weights)	w^2 (sum of squared weights)
Effect on weights	Drives some weights to exactly 0	Shrinks all weights, none exactly 0
Feature selection?	Yes (sparse models)	No (keeps all features)
When to use	Many features, suspect some are irrelevant	Many correlated features, want to keep all
Geometric intuition	Diamond-shaped constraint region	Circle-shaped constraint region

Common Similarity/Distance Metrics

Metric	Formula Intuition	Range	Best For
Euclidean (L2)	Straight-line distance	[0, inf)	When magnitude matters (image similarity)
Manhattan (L1)	Grid-walking distance	[0, inf)	Sparse, high-dimensional data
Cosine Similarity	Angle between vectors	[-1, 1]	Text/embeddings (ignores magnitude)
Dot Product	Projection of one vector onto another	(-inf, inf)	Attention mechanisms, unnormalized similarity

Optimizer Comparison

Property	SGD	SGD+Momentum	Adam	AdamW
Adaptive LR?	No	No	Yes	Yes
Weight Decay	Coupled	Coupled	Coupled (buggy)	Decoupled (correct)
Memory	O(n)	O(2n)	O(3n)	O(3n)
Best for	Fine-tuning with care	CNNs	General use	Transformers/LLMs
Default choice?	No	Sometimes	Often	Yes (for NLP)

8. Common Mistakes

Confusing correlation with causation: Correlation means two things move together. Causation means one CAUSES the other. Your model finding a correlation doesn't prove causation.

Forgetting that cosine similarity can be negative: Cosine ranges from -1 to 1, not 0 to 1. Negative means opposite directions. Some implementations clip to [0,1] for convenience.

Thinking gradient = 0 means minimum: It could be a maximum, saddle point, or plateau. In high-dimensional spaces, saddle points are far more common than local minima.

Using Euclidean distance for text embeddings: Text embeddings have high dimensionality and varying magnitudes. Cosine similarity is almost always better because it focuses on direction (meaning).

Confusing cross-entropy with KL divergence: Cross-entropy = H(p,q) = H(p) + KL(p||q). When the true distribution p is fixed, minimizing cross-entropy IS the same as minimizing KL divergence.

Applying softmax before cross-entropy manually: PyTorch's CrossEntropyLoss already includes softmax internally. Applying softmax first means you're doing it twice, which gives wrong gradients.

Not normalizing features before using L2 distance: L2 distance is sensitive to feature scales. If one feature ranges 0-1000 and another 0-1, the first dominates. Always normalize first.

Thinking a singular matrix is always a bug: In practice, near-singular (ill-conditioned) matrices cause numerical instability. This is why we use gradient descent instead of the Normal Equation for large problems.

9. Quick-Fire Q&A

25 frequently asked questions with concise answers. Use these for rapid revision.

[Easy] Q: What is a dot product and where is it used in AI?

A: Sum of element-wise products of two vectors. Used in attention (Q.K^T), similarity search, and every neural network layer (input @ weights).

[Medium] Q: Why do we use cosine similarity for text embeddings instead of Euclidean distance?

A: Cosine measures direction (meaning), Euclidean measures magnitude + direction. Text embeddings vary in magnitude based on sentence length, but direction captures semantic meaning.

[Easy] Q: What does the gradient of a function represent?

A: A vector pointing in the direction of steepest increase. Its magnitude tells you how steep the slope is. We move OPPOSITE to it in gradient descent.

[Medium] Q: Explain the chain rule in the context of backpropagation.

A: Backpropagation applies the chain rule to compute gradients layer by layer, from output to input. Each layer's gradient = local gradient * gradient from the layer above.

[Easy] Q: What is the difference between L1 and L2 regularization?

A: L1 adds |w| to loss, driving some weights to 0 (feature selection). L2 adds w^2, shrinking all weights evenly (no feature selection). L1 = sparse, L2 = small.

[Medium] Q: Why is cross-entropy used as a loss function for classification?

A: It heavily penalizes confident wrong predictions. Mathematically, minimizing cross-entropy = maximizing the likelihood of the training data (MLE).

[Medium] Q: What is an eigenvalue and why does it matter for PCA?

A: An eigenvalue tells you how much variance a principal component captures. The eigenvector with the largest eigenvalue = the direction of maximum variance in your data.

[Easy] Q: What is the Central Limit Theorem and why does it matter?

A: The average of many random samples is approximately normally distributed, regardless of the original distribution. This justifies Gaussian assumptions in many ML algorithms.

[Easy] Q: Explain the difference between convex and non-convex optimization.

A: Convex: single global minimum, gradient descent always finds it. Non-convex: multiple local minima and saddle points. Neural networks are non-convex, but good minima are reachable.

[Medium] Q: What is KL Divergence?

A: Measures how much one probability distribution differs from another. KL(p||q) >= 0, equals 0 only when distributions are identical. Not symmetric.

[Medium] Q: Why does Adam optimizer work better than plain SGD for most tasks?

A: Adam adapts the learning rate per-parameter using running averages of gradient and squared gradient. Parameters with large gradients get smaller steps, noisy parameters get bigger steps.

[Medium] Q: What is the softmax function and what does temperature do to it?

A: Softmax converts raw scores (logits) to probabilities that sum to 1. Temperature T scales logits before softmax: T<1 sharpens (more confident), T>1 flattens (more random).

[Medium] Q: What is the vanishing gradient problem?

A: In deep networks, gradients shrink exponentially as they're multiplied through many layers (chain rule). Layers near the input barely update. Solutions: ReLU activation, skip connections, batch norm.

[Easy] Q: Why do we need feature scaling?

A: Many algorithms (gradient descent, KNN, SVM) are sensitive to feature magnitudes. If one feature is 0-1000 and another is 0-1, the larger one dominates. Normalization puts them on equal footing.

[Hard] Q: What is MLE and how does it relate to neural network training?

A: Maximum Likelihood Estimation finds parameters that maximize P(data|model). Minimizing cross-entropy loss is mathematically equivalent to MLE.

[Medium] Q: What does the determinant of a matrix tell you?

A: It measures how much the matrix scales area/volume. Determinant = 0 means the matrix is singular (not invertible) and collapses some dimensions.

[Medium] Q: What is a covariance matrix?

A: A matrix where entry (i,j) is the covariance between feature i and feature j. The diagonal has variances. It's symmetric and positive semi-definite. Input to PCA.

[Hard] Q: Explain bias-variance tradeoff mathematically.

A: Error = Bias^2 + Variance + Irreducible noise. High bias = underfitting (model too simple). High variance = overfitting (model too complex). Sweet spot minimizes total error.

[Medium] Q: What is entropy in information theory?

A: Measures uncertainty in a distribution. H = -sum(p*log(p)). Maximum for uniform distribution, zero when outcome is certain. Used in decision trees (information gain).

[Easy] Q: Why is matrix multiplication not commutative?

A: A*B is not B*A in general because the operation depends on which rows multiply which columns. Even the shapes may not allow both: [2x3]*[3x4] works but [3x4]*[2x3] doesn't.

[Hard] Q: What is a saddle point and why is it relevant to deep learning?

A: A point where gradient = 0 but it's neither minimum nor maximum (minimum in some directions, maximum in others). In high dimensions, saddle points are far more common than local minima.

[Hard] Q: How does SVD relate to PCA?

A: PCA on centered data is equivalent to SVD. The right singular vectors (V) are principal components, and singular values squared / (n-1) are the eigenvalues of the covariance matrix.

[Easy] Q: What is a probability density function vs a probability mass function?

A: PMF is for discrete variables (gives exact probabilities). PDF is for continuous variables (gives probability density; actual probability requires integration over an interval).

[Medium] Q: Why do we use log probabilities instead of raw probabilities?

A: Multiplying many small probabilities causes numerical underflow (approaches 0). Log converts multiplication to addition, preventing underflow and making computation stable.

[Hard] Q: What is the role of the Jacobian in deep learning?

A: The Jacobian is the matrix of all partial derivatives for a vector function. In backprop, the chain rule involves multiplying Jacobians. PyTorch autograd handles this implicitly.

10. One-Line Cheat Sheet

Entire document compressed. One concept per line. Use for last-minute revision.

Dot Product — Multiply corresponding elements and sum. Measures similarity. Foundation of attention.

Cosine Similarity — Dot product normalized by magnitudes. The metric for embedding search and RAG.

L1 Norm — Sum of absolute values. Used in Lasso regularization for sparse models.

L2 Norm — Square root of sum of squares. Euclidean distance. Used in Ridge regularization.

Matrix Multiply — Inner dimensions must match. Every neural network layer is a matrix multiply.

Transpose — Flip rows and columns. Used in attention (Q*K^T), backprop, data reshaping.

Eigenvalues/vectors — Directions that don't change under transformation. PCA finds the top eigenvectors.

SVD — Factorizes any matrix into U*S*V^T. Backbone of PCA, recommender systems, compression.

Derivative — Rate of change. Positive = increasing, negative = decreasing, zero = critical point.

Gradient — Vector of all partial derivatives. Points toward steepest increase. We descend opposite.

Chain Rule — Derivative of composed functions = product of derivatives. THE rule behind backprop.

Jacobian — Matrix of all partial derivatives for vector functions. Implicit in autograd.

Probability Distribution — Function assigning probabilities to outcomes. Gaussian is the most important one.

Bayes' Theorem — P(A|B) = P(B|A)*P(A)/P(B). Update beliefs with evidence. Basis of Naive Bayes.

MLE — Find parameters that maximize P(data|model). Cross-entropy minimization = MLE.

Covariance/Correlation — Measure linear relationship between variables. Correlation is normalized to [-1, 1].

Central Limit Theorem — Average of many samples is approximately Gaussian. Justifies normality assumptions.

Gradient Descent — w = w - lr * gradient. Move opposite to gradient to minimize loss.

Learning Rate — Step size in gradient descent. Too large = diverge. Too small = stuck. Most important HP.

Adam Optimizer — Adaptive per-parameter learning rates using momentum + squared gradient. Default choice.

Convex vs Non-convex — Convex has one minimum (guaranteed). Non-convex has many (neural networks). Still works.

Entropy — Measures uncertainty. H = -sum(p*log(p)). High = uncertain, low = confident.

Cross-Entropy — Loss function for classification. Penalizes confident wrong predictions heavily.

KL Divergence — Measures distribution difference. KL >= 0. Used in VAEs, distillation, LLM training.

Temperature — Divides logits before softmax. T<1 = sharper (confident). T>1 = flatter (creative).

Graph — Nodes + edges. Agent workflows, knowledge graphs, computation graphs, GNNs.

DAG — Directed Acyclic Graph. PyTorch computation graph. ML pipeline structure.